there is a function, which converts WSG84-GPS coordinates to a mercator projection. How to map mercator projection coordinates to a x & y position in an application window?

/**

- @brief Converts WSG84-GPS coordinates to mercator projection coordinates, used for displaying
- @note Clamping latitude is important because the Mercator projection tends to infinity at the
- poles, and practical Web Mercator usage limits latitude to about 85.0511 degrees.
- @param scale
- @param longitude_deg in decimal degrees (will be converted to radians inside)
- @param latitude_deg in decimal degrees (will be converted to radians inside)
- @return
  */
  coordinates_t coordinates_mercator_project (double scale, double longitude_deg, double latitude_deg)
  {
  const double DEG_TO_RAD = M_PI / 180.0;
  const double MAX_LAT = 85.05112878; // practical Mercator limit

coordinates_t p;

if (latitude_deg > MAX_LAT)
latitude_deg = MAX_LAT;
if (latitude_deg < -MAX_LAT)
latitude_deg = -MAX_LAT;

double lon = longitude_deg * DEG_TO_RAD;
double lat = latitude_deg * DEG_TO_RAD;

p.longitude = (scale * lon);
p.latitude = (scale * log (tan (M_PI / 4.0 + lat / 2.0)));

return p;
}

Yes: after Mercator projection, you still need a second step that converts projected map coordinates into window coordinates using your current view rectangle and window size. In practice, this is just an affine transform: normalize projected coordinates within the visible Mercator bounds, then scale to pixel width and height, with Y usually inverted because screen coordinates grow downward.[1] [2]

Core idea

Your function returns Mercator-space coordinates, not screen-space coordinates. A screen position needs three more things: the Mercator coordinate of the current map center or visible bounds, the zoom/scale of the current view, and the window width and height in pixels.[2] [1]

If your window uses the usual top-left origin, then X increases to the right and Y increases downward, while Mercator Y increases upward, so the Y axis must be flipped during mapping. Libraries that project to screen pixels explicitly mention returning top-left pixel coordinates for canvas/SVG rendering.[2]

Simple formula

Assume you know the visible Mercator rectangle:

- `view_min_x`, `view_max_x`
- `view_min_y`, `view_max_y`

and the window size:

- `window_width`
- `window_height`

Then map a projected point `(mx, my)` to screen pixels like this:[1] [2]

```
double sx = (mx - view_min_x) / (view_max_x - view_min_x) * window_width;
double sy = (view_max_y - my) / (view_max_y - view_min_y) * window_height;
```

The X formula shifts the left edge to 0 and scales to the window width, while the Y formula uses `view_max_y - my` so north appears at the top of the screen. That matches the usual top-left window coordinate convention used by mapping/view libraries.[1] [2]

Center-based formula

Often it is easier to store the map center in Mercator coordinates instead of min/max bounds. If `(cx, cy)` is the Mercator center of the current view, then a point `(mx, my)` becomes:[2]

```
double sx = window_width  * 0.5 + (mx - cx) * pixels_per_mercator_unit;
double sy = window_height * 0.5 - (my - cy) * pixels_per_mercator_unit;
```

Here, `pixels_per_mercator_unit` is your zoom factor for drawing. The minus sign on Y is again the screen-axis flip.[2]

How to get the visible bounds

If your zoom is represented by "Mercator units per screen pixel" or the inverse, you can derive the visible bounds from the center. For example:[2]

```
double half_w = window_width  * 0.5 / pixels_per_mercator_unit;
double half_h = window_height * 0.5 / pixels_per_mercator_unit;

double view_min_x = cx - half_w;
double view_max_x = cx + half_w;
double view_min_y = cy - half_h;
double view_max_y = cy + half_h;
```

Then use the normalization formula from above. This is mathematically equivalent to the center-based form.[2]

With your function

Using your `coordinates_mercator_project`, a complete version could look like this:

```
typedef struct {
    double x;
    double y;
} screen_point_t;

screen_point_t mercator_to_screen(
    coordinates_t merc,
    double center_x,
    double center_y,
    double pixels_per_unit,
    int window_width,
    int window_height)
{
    screen_point_t s;
    s.x = window_width  * 0.5 + (merc.longitude - center_x) * pixels_per_unit;
    s.y = window_height * 0.5 - (merc.latitude  - center_y) * pixels_per_unit;
    return s;
}
```

And usage:

```
coordinates_t p = coordinates_mercator_project(scale, lon_deg, lat_deg);
screen_point_t s = mercator_to_screen(
    p,
    map_center_merc_x,
    map_center_merc_y,
    zoom_pixels_per_unit,
    window_width,
    window_height);
```

This works because the first function converts geographic coordinates into Mercator coordinates, and the second converts Mercator coordinates into top-left-origin window pixels relative to your current camera/view.[1] [2]

Important detail

Be careful not to use one `scale` value for two different meanings unless you intend that. In your projection code, `scale` multiplies the Mercator coordinates themselves, while in rendering you also need a view zoom factor that says how many pixels correspond to one Mercator unit; those can be the same by design, but conceptually they are different roles.[2]

Practical example

If your app window is `800x600`, your map center in Mercator space is `(1000, 500)`, and your point is `(1010, 490)` with `2` pixels per Mercator unit, then:[2]

- `x = 400 + (1010 - 1000) * 2 = 420`[2]
- `y = 300 - (490 - 500) * 2 = 320`[2]

So the point is drawn at `(420, 320)`.[2]

Recommendation

The cleanest design is:

1. Convert lon/lat to Mercator.
2. Store your current camera as Mercator center + zoom.
3. Convert Mercator to screen with translation, scaling, and Y flip.[1] [2]

If you want, I can also give you:

- a full `lat/lon -> screen x/y` function in C,
- support for panning and zooming,
- or the inverse `screen x/y -> lat/lon` function.

⁂

how these "Mercator units" corresponds to a distance (like in meters) in real world?

Mercator “units” are not directly meters; they are a scaled Easting/Northing that you must tie to your chosen map scale (and latitude) to get a meaningful distance in meters.[11] [12]

Concept: a scale factor

At the equator, Web Mercator is usually defined so that:

- the Earth's ellipsoid is approximated by a sphere of radius about $R \approx 6\,378\,137$ m, and
- the projected X unit is essentially "radians of longitude × R", i.e., $x = R\,\lambda$.[13] [11]

Then:

- 1 Mercator unit ≈ 1 meter at the equator, if you set your drawing scale such that "1 Mercator unit = 1 meter" there.[14] [11]
- If you instead use, say, `scale = 2.0` in your projection, then 1 Mercator unit corresponds to 0.5 m in real-world length at the equator (because everything is stretched horizontally by 2×).[15] [14]

So in general:

$$\text{distance in meters} \approx (\text{Mercator units}) \times \text{scale_factor}^{-1}$$

where `scale_factor` is the factor you pass into your projection (or later into your viewport) that links projected Mercator coordinates to meters at the reference latitude.[16] [17]

Latitude dependence

Because Mercator stretches vertically with latitude, the number of meters per Mercator unit depends on where you measure it.[18] [19]

- At higher latitudes, the scale factor increases roughly by $\sec\varphi$ (1 / cos φ), so:
  - 1 horizontal Mercator unit at latitude φ ≈ 1 m × $R/R_{\text{eq}} \times \sec\varphi$
  - in practice, at 60°, 1 Mercator unit corresponds to about 2 m along a meridian, while at the equator it's about 1 m per unit.[12] [18]

If you want precise "distance in meters" from Mercator differences, you must:

1. choose a reference latitude (often the map center),
2. compute the Mercator scale factor at that latitude,
3. multiply Mercator distances (Δx, Δy) by scale_factor^{-1}.[12] [16]

Practical rule-of-thumb

If you set your `scale` so that:

- at your target latitude, "1 Mercator unit" ≈ 1 m in your map,
- then the Euclidean distance between two Mercator points is approximately the distance in meters, assuming you stay near that latitude and the map is not too large.[17] [12]

If you tell me your `scale` and the latitude you care about, I can derive an explicit "1 Mercator unit = X meters" factor for your app.

⁂

ok. go one prompt back and give a full lat/lon -> screen x/y function in C

Sure — below is a complete C function that goes directly from latitude/longitude to screen `x`, `y`, using the same Mercator math as your function and a map center + zoom in Mercator units. The key steps are: clamp latitude to the practical Mercator limit, project lon/lat into Mercator coordinates, subtract the current map center, scale into pixels, and flip Y so north appears upward while screen Y grows downward.[26] [27]

Full function

```
#include <math.h>

typedef struct
{
    double x;
    double y;
} screen_point_t;

typedef struct
{
    double x;   /* mercator x */
    double y;   /* mercator y */
} mercator_point_t;

/* Convert WGS84 lon/lat in degrees to Mercator coordinates */
static mercator_point_t
wgs84_to_mercator(double mercator_scale, double longitude_deg, double latitude_deg)
{
    const double DEG_TO_RAD = M_PI / 180.0;
    const double MAX_LAT = 85.05112878;

    if (latitude_deg > MAX_LAT)  latitude_deg = MAX_LAT;
    if (latitude_deg < -MAX_LAT) latitude_deg = -MAX_LAT;

    double lon = longitude_deg * DEG_TO_RAD;
    double lat = latitude_deg  * DEG_TO_RAD;

    mercator_point_t p;
    p.x = mercator_scale * lon;
    p.y = mercator_scale * log(tan(M_PI / 4.0 + lat / 2.0));
    return p;
}

/*
 * Convert WGS84 lon/lat directly to screen coordinates.
 *
 * Parameters:
 *   mercator_scale         - scale used inside Mercator projection
 *   longitude_deg          - point longitude in degrees
 *   latitude_deg           - point latitude in degrees
 *   center_longitude_deg   - map center longitude in degrees
```

```
 *   center_latitude_deg    - map center latitude in degrees
 *   pixels_per_mercator    - zoom: how many screen pixels per Mercator unit
 *   window_width           - window width in pixels
 *   window_height          - window height in pixels
 *
 * Returns:
 *   screen x,y in pixels, with origin at top-left
 */
screen_point_t
latlon_to_screen_xy(double mercator_scale,
                    double longitude_deg,
                    double latitude_deg,
                    double center_longitude_deg,
                    double center_latitude_deg,
                    double pixels_per_mercator,
                    int window_width,
                    int window_height)
{
    mercator_point_t p = wgs84_to_mercator(mercator_scale, longitude_deg, latitude_
    mercator_point_t c = wgs84_to_mercator(mercator_scale, center_longitude_deg, ce

    screen_point_t s;
    s.x = (window_width  * 0.5) + (p.x - c.x) * pixels_per_mercator;
    s.y = (window_height * 0.5) - (p.y - c.y) * pixels_per_mercator;

    return s;
}
```

This version uses the center of the window as the map center, which is a common camera model for 2D map rendering. The Y expression subtracts Mercator Y from the screen center because projected north is positive upward, but screen coordinates usually increase downward.[27] [26]

Example usage

```
screen_point_t pt = latlon_to_screen_xy(
    1.0,        /* mercator_scale */
    19.0402,    /* point lon  */
    49.8224,    /* point lat  */
    19.0000,    /* map center lon */
    49.8000,    /* map center lat */
    50000.0,    /* pixels per mercator unit */
    1280,       /* window width */
    720         /* window height */
);
```

After that, `pt.x` and `pt.y` are the pixel coordinates where you draw the point in your application window. If the point is outside the current viewport, the returned coordinates will naturally be outside `[0,width]` or `[0,height]`.[28] [26]

If you want pan offsets

If your renderer uses a top-left map origin instead of a centered camera, you can convert through explicit pan offsets:

```
screen_x = (p.x * pixels_per_mercator) + pan_x;
screen_y = (-p.y * pixels_per_mercator) + pan_y;
```

That is the same math, just expressed with a translation vector instead of a center coordinate. The centered-camera version is usually easier to reason about for interactive zooming and panning.[26] [28]

Important note

Your `mercator_scale` and `pixels_per_mercator` are different concepts even though both affect size on screen. `mercator_scale` defines the size of the projected coordinate system itself, while `pixels_per_mercator` is your viewport zoom that converts projected coordinates into screen pixels.[27] [28]

If you want, I can next give you one of these:

- a version that uses a bounding box instead of map center,
- a matching `screen x/y -> lat/lon` inverse function,
- or a version adapted to SDL/OpenGL.

⁂

give me a matching screen x/y -> lat/lon inverse function

Yes — the inverse is just the reverse of the forward chain:

1. screen `x, y` → Mercator `x, y`
2. Mercator `x, y` → lon/lat[36] [37]

For spherical/Web Mercator, the inverse formulas are:

- $\lambda = x/s$
- $\phi = 2\arctan(\exp(y/s)) - \pi/2$

where `s` is the Mercator scale you used in your projection function.[38] [37]

Matching inverse in C

This version matches the forward function I gave before: same center-based camera, same `mercator_scale`, same `pixels_per_mercator`, and top-left screen origin.[37] [36]

```c
#include <math.h>

typedef struct
{
    double longitude_deg;
    double latitude_deg;
} geo_point_t;

typedef struct
{
    double x;
    double y;
} mercator_point_t;

/* Forward helper: same as before */
static mercator_point_t
wgs84_to_mercator(double mercator_scale, double longitude_deg, double latitude_deg)
{
    const double DEG_TO_RAD = M_PI / 180.0;
    const double MAX_LAT = 85.05112878;

    if (latitude_deg > MAX_LAT)  latitude_deg = MAX_LAT;
    if (latitude_deg < -MAX_LAT) latitude_deg = -MAX_LAT;

    double lon = longitude_deg * DEG_TO_RAD;
    double lat = latitude_deg  * DEG_TO_RAD;

    mercator_point_t p;
    p.x = mercator_scale * lon;
    p.y = mercator_scale * log(tan(M_PI / 4.0 + lat / 2.0));
    return p;
}

/* Inverse helper: Mercator -> WGS84 */
static geo_point_t
mercator_to_wgs84(double mercator_scale, double mx, double my)
{
    const double RAD_TO_DEG = 180.0 / M_PI;
    const double MAX_LAT = 85.05112878;

    geo_point_t g;

    double lon_rad = mx / mercator_scale;
    double lat_rad = 2.0 * atan(exp(my / mercator_scale)) - M_PI / 2.0;

    g.longitude_deg = lon_rad * RAD_TO_DEG;
    g.latitude_deg  = lat_rad * RAD_TO_DEG;

    if (g.latitude_deg > MAX_LAT)  g.latitude_deg = MAX_LAT;
    if (g.latitude_deg < -MAX_LAT) g.latitude_deg = -MAX_LAT;

    return g;
}

/*
 * Convert screen x/y back to WGS84 lon/lat.
```

```
 *
 * Parameters:
 *   mercator_scale         - same scale used in forward Mercator projection
 *   screen_x               - screen x in pixels
 *   screen_y               - screen y in pixels
 *   center_longitude_deg   - map center longitude in degrees
 *   center_latitude_deg    - map center latitude in degrees
 *   pixels_per_mercator    - same zoom used in forward transform
 *   window_width           - window width in pixels
 *   window_height          - window height in pixels
 *
 * Returns:
 *   longitude/latitude in degrees
 */
geo_point_t
screen_xy_to_latlon(double mercator_scale,
                    double screen_x,
                    double screen_y,
                    double center_longitude_deg,
                    double center_latitude_deg,
                    double pixels_per_mercator,
                    int window_width,
                    int window_height)
{
    mercator_point_t c = wgs84_to_mercator(mercator_scale,
                                           center_longitude_deg,
                                           center_latitude_deg);

    mercator_point_t p;
    p.x = c.x + (screen_x - window_width  * 0.5) / pixels_per_mercator;
    p.y = c.y - (screen_y - window_height * 0.5) / pixels_per_mercator;

    return mercator_to_wgs84(mercator_scale, p.x, p.y);
}
```

The X inversion adds the screen offset back to the center Mercator X, while the Y inversion subtracts the screen offset because the forward transform flipped Y. After that, the standard inverse Mercator formulas recover longitude and latitude.[36] [37]

Why the signs look like that

In the forward transform, you used:

```
screen_x = cx_screen + (mx - cx_merc) * pixels_per_mercator;
screen_y = cy_screen - (my - cy_merc) * pixels_per_mercator;
```

So solving for `mx` and `my` gives:

```
mx = cx_merc + (screen_x - cx_screen) / pixels_per_mercator;
my = cy_merc - (screen_y - cy_screen) / pixels_per_mercator;
```

That is exactly what the inverse function implements.[38] [37]

Example usage

```
geo_point_t g = screen_xy_to_latlon(
    1.0,        /* mercator_scale */
    mouse_x,     /* screen x */
    mouse_y,     /* screen y */
    19.0000,    /* map center lon */
    49.8000,    /* map center lat */
    50000.0,    /* pixels per mercator */
    1280,       /* window width */
    720         /* window height */
);
```

This is useful for mouse picking, cursor readout, placing markers, or converting a clicked pixel back into geographic coordinates.[39] [40]

Matching forward function

For convenience, here is the forward one again in a compatible form:

```
typedef struct
{
    double x;
    double y;
} screen_point_t;

screen_point_t
latlon_to_screen_xy(double mercator_scale,
                    double longitude_deg,
                    double latitude_deg,
                    double center_longitude_deg,
                    double center_latitude_deg,
                    double pixels_per_mercator,
                    int window_width,
                    int window_height)
{
    mercator_point_t p = wgs84_to_mercator(mercator_scale, longitude_deg, latitude_
    mercator_point_t c = wgs84_to_mercator(mercator_scale, center_longitude_deg, ce

    screen_point_t s;
    s.x = (window_width  * 0.5) + (p.x - c.x) * pixels_per_mercator;
    s.y = (window_height * 0.5) - (p.y - c.y) * pixels_per_mercator;
    return s;
}
```

These two functions form a matched pair as long as you pass the same center, window size, Mercator scale, and zoom factor into both.[37] [36]

Small caution

If `pixels_per_mercator` is zero or extremely small, the inverse becomes invalid or numerically unstable, so guard against that in production code. Also, longitude may naturally fall outside `[-180, 180]` if you allow wrapping; you can normalize it afterward if needed.[40] [37]

If you want, I can combine all of this into one small C header with:

- structs,
- forward + inverse functions,
- longitude normalization,
- and meters-per-pixel helper functions.

⁂

yes, combine all of this into one small C header with everytyhing

Below is a small self-contained C header that bundles the full set: WGS84 lon/lat ↔ Mercator, lon/lat ↔ screen x/y, longitude normalization, latitude clamping, and helpers for approximate meters-per-Mercator-unit and meters-per-pixel at a given latitude. The Mercator forward and inverse formulas are the standard spherical/Web Mercator ones, and the ground-resolution helper follows the usual latitude-dependent cosine relationship used in Web Mercator scale calculations.[46] [47] [48]

Header

```
#ifndef SIMPLE_MERCATOR_H
#define SIMPLE_MERCATOR_H

#include <math.h>

#ifdef __cplusplus
extern "C" {
#endif

/*
 * Small WGS84/Web-Mercator utility header
 *
 * Coordinate model:
 * - Geographic coordinates are longitude/latitude in degrees.
 * - Mercator coordinates are projected coordinates:
 *       x = scale * lon_rad
 *       y = scale * log(tan(pi/4 + lat_rad/2))
 * - Screen coordinates assume:
 *       origin at top-left,
 *       +x to the right,
 *       +y downward,
 *       map center positioned at the window center.
 *
```

```
 * Notes:
 * - This is spherical/Web Mercator style math.
 * - Mercator latitude is clamped to +/- 85.05112878 degrees.
 * - `mercator_scale` defines the size of the Mercator coordinate system.
 * - `pixels_per_mercator` defines viewport zoom.
 */

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

#define MERCATOR_MAX_LAT_DEG 85.05112878
#define WGS84_EARTH_RADIUS_M 6378137.0
#define WGS84_EARTH_CIRCUMFERENCE_M (2.0 * M_PI * WGS84_EARTH_RADIUS_M)

typedef struct
{
    double longitude_deg;
    double latitude_deg;
} sm_geo_point_t;

typedef struct
{
    double x;
    double y;
} sm_mercator_point_t;

typedef struct
{
    double x;
    double y;
} sm_screen_point_t;

/* -------------------------- basic helpers -------------------------- */

static inline double sm_deg_to_rad(double deg)
{
    return deg * (M_PI / 180.0);
}

static inline double sm_rad_to_deg(double rad)
{
    return rad * (180.0 / M_PI);
}

static inline double sm_clamp_latitude_deg(double latitude_deg)
{
    if (latitude_deg > MERCATOR_MAX_LAT_DEG)  return MERCATOR_MAX_LAT_DEG;
    if (latitude_deg < -MERCATOR_MAX_LAT_DEG) return -MERCATOR_MAX_LAT_DEG;
    return latitude_deg;
}

static inline double sm_normalize_longitude_deg(double longitude_deg)
{
    double x = fmod(longitude_deg + 180.0, 360.0);
    if (x < 0.0)
```

```
        x += 360.0;
    return x - 180.0;
}

/* Optional: wrap to [0, 360) instead of [-180, 180) */
static inline double sm_normalize_longitude_deg_360(double longitude_deg)
{
    double x = fmod(longitude_deg, 360.0);
    if (x < 0.0)
        x += 360.0;
    return x;
}

/* -------------------------- mercator <-> geo -------------------------- */

static inline sm_mercator_point_t
sm_wgs84_to_mercator(double mercator_scale,
                     double longitude_deg,
                     double latitude_deg)
{
    double lon_rad = sm_deg_to_rad(longitude_deg);
    double lat_rad = sm_deg_to_rad(sm_clamp_latitude_deg(latitude_deg));

    sm_mercator_point_t p;
    p.x = mercator_scale * lon_rad;
    p.y = mercator_scale * log(tan(M_PI / 4.0 + lat_rad / 2.0));
    return p;
}

static inline sm_geo_point_t
sm_mercator_to_wgs84(double mercator_scale,
                     double mercator_x,
                     double mercator_y)
{
    sm_geo_point_t g;

    double lon_rad = mercator_x / mercator_scale;
    double lat_rad = 2.0 * atan(exp(mercator_y / mercator_scale)) - (M_PI / 2.0);

    g.longitude_deg = sm_rad_to_deg(lon_rad);
    g.latitude_deg  = sm_rad_to_deg(lat_rad);

    g.longitude_deg = sm_normalize_longitude_deg(g.longitude_deg);
    g.latitude_deg  = sm_clamp_latitude_deg(g.latitude_deg);

    return g;
}

/* -------------------------- geo <-> screen -------------------------- */

static inline sm_screen_point_t
sm_latlon_to_screen_xy(double mercator_scale,
                       double longitude_deg,
                       double latitude_deg,
                       double center_longitude_deg,
                       double center_latitude_deg,
```

```
                      double pixels_per_mercator,
                      int window_width,
                      int window_height)
{
    sm_mercator_point_t p = sm_wgs84_to_mercator(
        mercator_scale, longitude_deg, latitude_deg);

    sm_mercator_point_t c = sm_wgs84_to_mercator(
        mercator_scale, center_longitude_deg, center_latitude_deg);

    sm_screen_point_t s;
    s.x = (window_width  * 0.5) + (p.x - c.x) * pixels_per_mercator;
    s.y = (window_height * 0.5) - (p.y - c.y) * pixels_per_mercator;

    return s;
}

static inline sm_geo_point_t
sm_screen_xy_to_latlon(double mercator_scale,
                      double screen_x,
                      double screen_y,
                      double center_longitude_deg,
                      double center_latitude_deg,
                      double pixels_per_mercator,
                      int window_width,
                      int window_height)
{
    sm_mercator_point_t c = sm_wgs84_to_mercator(
        mercator_scale, center_longitude_deg, center_latitude_deg);

    sm_mercator_point_t p;
    p.x = c.x + (screen_x - window_width  * 0.5) / pixels_per_mercator;
    p.y = c.y - (screen_y - window_height * 0.5) / pixels_per_mercator;

    return sm_mercator_to_wgs84(mercator_scale, p.x, p.y);
}

/* -------------------------- mercator <-> screen -------------------------- */

static inline sm_screen_point_t
sm_mercator_to_screen_xy(double mercator_x,
                        double mercator_y,
                        double center_mercator_x,
                        double center_mercator_y,
                        double pixels_per_mercator,
                        int window_width,
                        int window_height)
{
    sm_screen_point_t s;
    s.x = (window_width  * 0.5) + (mercator_x - center_mercator_x) * pixels_per_merc
    s.y = (window_height * 0.5) - (mercator_y - center_mercator_y) * pixels_per_merc
    return s;
}

static inline sm_mercator_point_t
sm_screen_xy_to_mercator(double screen_x,
```

```
                        double screen_y,
                        double center_mercator_x,
                        double center_mercator_y,
                        double pixels_per_mercator,
                        int window_width,
                        int window_height)
{
    sm_mercator_point_t p;
    p.x = center_mercator_x + (screen_x - window_width  * 0.5) / pixels_per_mercator
    p.y = center_mercator_y - (screen_y - window_height * 0.5) / pixels_per_mercator
    return p;
}

/* -------------------------- distance / scale helpers -------------------------- */

/*
 * Approximate meters represented by 1 degree longitude at latitude.
 * Useful as a sanity check, not as a Mercator-space metric.
 */
static inline double sm_meters_per_degree_longitude(double latitude_deg)
{
    double lat_rad = sm_deg_to_rad(sm_clamp_latitude_deg(latitude_deg));
    return (WGS84_EARTH_CIRCUMFERENCE_M * cos(lat_rad)) / 360.0;
}

/*
 * Approximate meters represented by 1 degree latitude.
 * Good rough Earth-scale approximation.
 */
static inline double sm_meters_per_degree_latitude(void)
{
    return WGS84_EARTH_CIRCUMFERENCE_M / 360.0;
}

/*
 * Approximate meters per 1 Mercator unit at the given latitude.
 *
 * Since:
 *   mercator_x = mercator_scale * lon_rad
 * and one radian at latitude phi is R * cos(phi) meters on the ground,
 * then:
 *   meters_per_mercator_unit ~= (R * cos(phi)) / mercator_scale
 *
 * This is a local east-west ground scale. In Web Mercator, local scale
 * distortion is isotropic, so this is also the local meters-per-unit
 * factor for small distances around that latitude.
 */
static inline double sm_meters_per_mercator_unit(double mercator_scale,
                                                 double latitude_deg)
{
    double lat_rad = sm_deg_to_rad(sm_clamp_latitude_deg(latitude_deg));
    return (WGS84_EARTH_RADIUS_M * cos(lat_rad)) / mercator_scale;
}

/*
 * Approximate Mercator units per meter at the given latitude.
```

```
 */
static inline double sm_mercator_units_per_meter(double mercator_scale,
                                                 double latitude_deg)
{
    double mpu = sm_meters_per_mercator_unit(mercator_scale, latitude_deg);
    return (mpu != 0.0) ? (1.0 / mpu) : 0.0;
}

/*
 * Approximate meters per screen pixel at the given latitude.
 *
 * If one Mercator unit corresponds locally to:
 *   sm_meters_per_mercator_unit(...)
 * and you draw:
 *   pixels_per_mercator pixels per Mercator unit
 * then:
 *   meters_per_pixel = meters_per_mercator_unit / pixels_per_mercator
 */
static inline double sm_meters_per_pixel(double mercator_scale,
                                         double pixels_per_mercator,
                                         double latitude_deg)
{
    if (pixels_per_mercator == 0.0)
        return 0.0;

    return sm_meters_per_mercator_unit(mercator_scale, latitude_deg) /
           pixels_per_mercator;
}

/*
 * Approximate screen pixels per meter at the given latitude.
 */
static inline double sm_pixels_per_meter(double mercator_scale,
                                         double pixels_per_mercator,
                                         double latitude_deg)
{
    double mpp = sm_meters_per_pixel(mercator_scale, pixels_per_mercator, latitude
    return (mpp != 0.0) ? (1.0 / mpp) : 0.0;
}

/* -------------------------- visibility helpers -------------------------- */

static inline int sm_screen_point_is_inside(sm_screen_point_t p,
                                            int window_width,
                                            int window_height)
{
    return (p.x >= 0.0 && p.x < (double)window_width &&
            p.y >= 0.0 && p.y < (double)window_height);
}

#ifdef __cplusplus
}
#endif

#endif /* SIMPLE_MERCATOR_H */
```

The forward and inverse geographic transforms use the standard Mercator equations, while the screen conversion is the same center-based viewport mapping discussed earlier: translate relative to map center, scale by pixels-per-Mercator-unit, and invert Y for top-left-origin screens.[49] [47] [48]

Minimal example

```
#include <stdio.h>
#include "simple_mercator.h"

int main(void)
{
    double mercator_scale = WGS84_EARTH_RADIUS_M;
    double pixels_per_mercator = 0.002;

    double center_lon = 19.0000;
    double center_lat = 49.8000;

    int w = 1280;
    int h = 720;

    sm_screen_point_t s = sm_latlon_to_screen_xy(
        mercator_scale,
        19.0402, 49.8224,
        center_lon, center_lat,
        pixels_per_mercator,
        w, h
    );

    printf("screen: x=%f y=%f\n", s.x, s.y);

    sm_geo_point_t g = sm_screen_xy_to_latlon(
        mercator_scale,
        s.x, s.y,
        center_lon, center_lat,
        pixels_per_mercator,
        w, h
    );

    printf("geo: lon=%f lat=%f\n", g.longitude_deg, g.latitude_deg);

    printf("meters/pixel near center: %f\n",
           sm_meters_per_pixel(mercator_scale, pixels_per_mercator, center_lat));

    return 0;
}
```

If you set `mercator_scale = WGS84_EARTH_RADIUS_M`, then your Mercator X/Y are in the common "meters-like" Web Mercator form $x = R\lambda$, $y = R\ln(\tan(\pi/4 + \phi/2))$, and the local meters-per-pixel helper becomes easy to interpret at a chosen latitude.[50] [46]

Notes

`sm_meters_per_mercator_unit()` and `sm_meters_per_pixel()` are local approximations, not globally exact distances, because Mercator distorts scale with latitude; the cosine term captures that latitude dependence in the usual Web Mercator ground-resolution formula.[51] [46]

If you want, I can also give you a second version of this header that uses:

- explicit pan offsets instead of center lon/lat,
- a bounding-box viewport,
- or float-only types for embedded use.

⁂

1. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-utils
2. https://en.wikipedia.org/wiki/Transverse_Mercator_projection
3. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-viewport
4. https://www.mapwindow.org/mapwingis/group__map__coordinates.html
5. https://deck.gl/docs/api-reference/core/web-mercator-viewport
6. https://tangrams.readthedocs.io/en/main/Overviews/Projections-Overview/
7. https://gambaswiki.org/wiki/comp/gb.opengl.glu/glu/project?l=fr&nh
8. https://stackoverflow.com/questions/64937112/map-web-mercator-pixel-coordinates-to-screen-pixel
9. https://stackoverflow.com/questions/14329691/convert-latitude-longitude-point-to-a-pixels-x-y-on-mercator-projection
10. https://pro.arcgis.com/en/pro-app/latest/help/mapping/properties/coordinate-systems-and-projections.htm
11. https://gist.github.com/phamquyhai/92d95154c45ddea6fe1902e8824a22fd
12. https://docs.geodesk.com/python/mercator
13. https://app.unpkg.com/maplibre-gl@5.2.0/files/src/geo/mercator_coordinate.ts
14. https://surferhelp.goldensoftware.com/projections/Mercator_Projection.htm
15. https://surferhelp.goldensoftware.com/projections/Transverse_Mercator_Projection.htm
16. https://cdn.jsdelivr.net/npm/maplibre-gl@5.2.0/src/geo/mercator_coordinate.ts
17. https://api.visicom.ua/en/products/tiles/theoretical-info
18. https://infogalactic.com/info/Mercator_projection
19. https://en.wikipedia.org/wiki/Mercator_projection
20. https://www.calculatorsconversion.com/en/calculator-coordinates-converter-must-have-best-tool/
21. https://proj.org/en/stable/operations/projections/merc.html
22. https://ccv.eng.wayne.edu/reference/mercator-15dec2015.pdf
23. https://tchayen.com/web-mercator-projection

24. https://hypar-io.github.io/Elements/api/Elements.Spatial.MercatorProjection.html
25. https://stackoverflow.com/questions/26406151/d3-mercator-projection-measuring-distance-or-converting-in-meters
26. https://stackoverflow.com/questions/64937112/map-web-mercator-pixel-coordinates-to-screen-pixel
27. https://en.wikipedia.org/wiki/Web_Mercator_projection
28. https://www.npmjs.com/package/viewport-mercator-project/v/4.0.0-alpha.3
29. https://en.wikipedia.org/wiki/Mercator_projection
30. https://stackoverflow.com/questions/14329691/convert-latitude-longitude-point-to-a-pixels-x-y-on-mercator-projection
31. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-utils
32. https://www.reddit.com/r/learnprogramming/comments/1qeeq2/c_converting_latlong_to_pixels_on_a_mercator/
33. https://api.kde.org/marble/html/MercatorProjection_8h_source.html
34. https://github.com/uber-archive/viewport-mercator-project/blob/master/docs/api-reference/web-mercator-viewport.md
35. https://gist.github.com/aaronhoffman/f53e1852ca289e4a806c062f97a18f05
36. https://www.chrisjones.id.au/Projection/index.html
37. https://en.wikipedia.org/wiki/Mercator_projection
38. https://epsg.io/1026-method
39. https://stackoverflow.com/questions/47498309/inverting-web-mercator-equations-to-convert-pixels-to-latitude-longitude
40. https://help.openstreetmap.org/questions/75611/transform-xy-pixel-values-into-lat-and-long/
41. https://proj.org/en/stable/operations/projections/merc.html
42. https://copyprogramming.com/howto/inverting-web-mercator-equations-to-convert-pixels-to-latitude-longitude
43. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-utils
44. https://proj.org/en/9.2/operations/projections/merc.html
45. https://chrisjones.id.au/Projection/
46. https://gist.github.com/fgcarto/a24d89067f15551c622c
47. https://en.wikipedia.org/wiki/Mercator_projection
48. https://epsg.io/1026-method
49. https://stackoverflow.com/questions/47498309/inverting-web-mercator-equations-to-convert-pixels-to-latitude-longitude
50. https://proj.org/en/stable/operations/projections/merc.html
51. https://gist.github.com/perrygeo/4478844
52. https://copyprogramming.com/howto/inverting-web-mercator-equations-to-convert-pixels-to-latitude-longitude

53. https://stackoverflow.com/questions/30326797/math-behind-mkmeterspermappointatlatitude
54. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-utils
55. https://api.visicom.ua/en/products/tiles/theoretical-info
56. https://www.php.cn/faq/1796674483.html
57. https://stackoverflow.com/questions/14329691/convert-latitude-longitude-point-to-a-pixels-x-y-on-mercator-projection
58. https://gist.github.com/skyebook/3152659